

THAT'S IMPOSSIBLE!

NOT MUCH EASIER...



 Dataedo /cartoon

# Introduction to Database Systems

## Spring 2025, Lecture 4:

### SQL Queries

# Agenda

## Select

- Renaming
- Expressions, casts, constants
- Distinct
- Filtering rows (**where**)
- Null values, **IN**, **LIKE**
- Sorting and limit queries

## Aggregation

- Aggregate Functions
- Grouping and filtering groups (**having**)

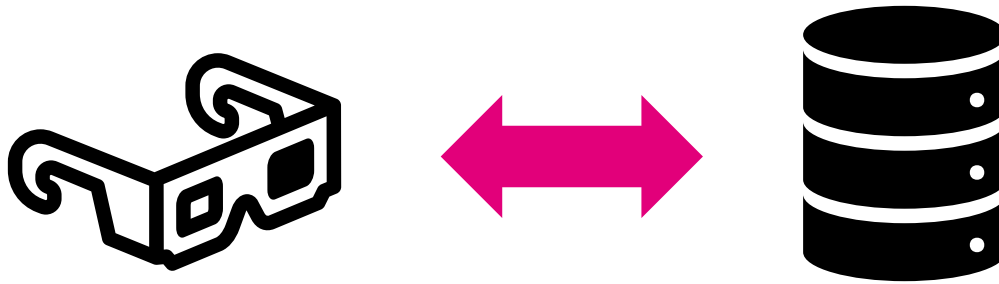
## Joins

- Semantics and syntax

# Intended Learning Outcomes

1. Suggest a database design in the ER model and convert to a relational database schema in a suitable normal form.
2. Write SQL queries, involving multiple relations, compound conditions, grouping, aggregation, and subqueries.
3. Use relational DBMSs from a conventional programming language in a secure manner.
4. Analyze/predict/improve query processing efficiency of the designed database using indices.
5. Reflect upon the evolution of the hardware and storage hierarchy and its impact on data management system design.
6. Discuss the pros and cons of different classes of data systems for modern analytics and data science applications.

2. Write SQL queries, involving multiple relations, compound conditions, grouping, aggregation, and subqueries.



Work with the data stored in the database

# Coffeehouse Database

**Shift** (Day, Cashier)

| Day    | Cashier |
|--------|---------|
| Feb 11 | Mads    |
| Feb 12 | Anna    |
| Feb 18 | Anna    |
| Feb 19 | Anna    |

**User** (UserName, Email, Password)

| UserName | Email | Password |
|----------|-------|----------|
| Anna     | anna  | ***      |
| Martin   | mhent | ***      |
| Omar     | omsh  | ***      |

**Product** (ProductName, Price)

| ProductName | Price |
|-------------|-------|
| Tea         | 100   |
| Small       | 85    |
| Large       | 100   |

**Purchase** (PurchaseTime, ProductName, UserName)

| PurchaseTime  | ProductName | UserName |
|---------------|-------------|----------|
| Feb 11, 9:55  | Tea         | Martin   |
| Feb 12, 10:03 | Small       | Martin   |
| Feb 12, 10:05 | Small       | Omar     |
| Feb 12, 10:06 | Large       | Omar     |
| Feb 19, 9:00  | Small       | Martin   |

## Attribute types

```
create table shift(day date, cashier string);  
create table user(username string, email string, password string);  
create table product(productname string, price int);  
create table purchase(purchasetime timestamp, productname string,  
                      username string);
```

All SQL queries of this lecture can be downloaded from  
LearnIT

- lecture4.sql

# Agenda

## Select

- Renaming
- Expressions, casts, constants
- Distinct
- Filtering rows (**where**)
- Null values, **IN**, **LIKE**
- Sorting and limit queries

## Aggregation

- Aggregate Functions
- Grouping and filtering groups (**having**)

## Joins

- Semantics and syntax

# Select Statements

Basic select statements are of the form:

```
select  -- list of attributes
from    -- one or more relations
where   -- filters
```

Result:

- A relation (table) or a single attribute
- Multiset semantics = duplicates are possible
- Not sorted if no **order by** is specified

Note: SQL is case-insensitive. Keywords, table names, and attribute names can be written in uppercase or lowercase. However, string comparisons are case-sensitive.

# Select

Select everything from a table.

SQL Query:

```
-- list all users
```

```
select * from user;
```

Comment

Table

\* is short for:  
all attributes  
(all columns)

Result:

| A-Z username ▼ | A-Z email ▼ | A-Z password ▼ |
|----------------|-------------|----------------|
| Anna           | anna        | ***            |
| Martin         | mhent       | ***            |
| Omar           | omsh        | ***            |

User (UserName, Email, Password)

| UserName | Email | Password |
|----------|-------|----------|
| Anna     | anna  | ***      |
| Martin   | mhent | ***      |
| Omar     | omsh  | ***      |

# Select Specific Columns

## SQL Query:

```
-- list username and email  
select username, email  
from user;
```

## Result:

| A-Z username ▼ | A-Z email ▼ |
|----------------|-------------|
| Anna           | anna        |
| Martin         | mhent       |
| Omar           | omsh        |

User (UserName, Email, Password)

| UserName | Email | Password |
|----------|-------|----------|
| Anna     | anna  | ***      |
| Martin   | mhent | ***      |
| Omar     | omsh  | ***      |

List of attributes,  
separated by  
commas.

Remember: result  
is not sorted. It can  
be in any order.

# Rename Columns

```
-- rename username to name  
select username as name, email  
from user;
```

Result:

| A-Z name ▼ | A-Z email ▼ |
|------------|-------------|
| Anna       | anna        |
| Martin     | mhent       |
| Omar       | omsh        |

User (UserName, Email, Password)

| UserName | Email | Password |
|----------|-------|----------|
| Anna     | anna  | ***      |
| Martin   | mhent | ***      |
| Omar     | omsh  | ***      |

The “as” keyword allows to rename column names.

# Rename Tables

User (UserName, Email, Password)

| UserName | Email | Password |
|----------|-------|----------|
| Anna     | anna  | ***      |
| Martin   | mhent | ***      |
| Omar     | omsh  | ***      |

```
-- rename table user to u  
select u.username, u.email  
from user as u;
```

Result:

| A-Z username ▼ | A-Z email ▼ |
|----------------|-------------|
| Anna           | anna        |
| Martin         | mhent       |
| Omar           | omsh        |

The “as” keyword allows to also rename table names. Attributes can be prefixed by table names (or their renamed names).

```
-- shorter  
select u.username, u.email  
from user u;
```

The “as” keyword can be omitted (leading to the same result).

# Expressions in Select

```
-- double the price of products  
select productname,  
price * 2 as doublePrice  
from product;
```

Result:

| A-Z productname ▼ | 123 doublePrice ▼ |
|-------------------|-------------------|
| Tea               | 200               |
| Small             | 170               |
| Large             | 200               |

**Product** (ProductName, Price)

| ProductName | Price |
|-------------|-------|
| Tea         | 100   |
| Small       | 85    |
| Large       | 100   |

Expressions can be anything, for example math (like here), function calls, casts.

# Casts

## Expression special case: casts

- Casts convert a value in a particular data type to the corresponding value in another data type
- Either with **cast()** function or with **::**
- Sometimes implicit (e.g. in comparisons)

```
-- cast price to the double type
select productname,
cast(price as double)
from product;
```

| productname | CST(price AS DOUBLE) |
|-------------|----------------------|
| Tea         | 100.0                |
| Small       | 85.0                 |
| Large       | 100.0                |

```
-- cast purchase time to date
-- and then to string
select purchasetime::date::string as str,
username, productname
from purchase;
```

| A-Z str    | A-Z username | A-Z productname |
|------------|--------------|-----------------|
| 2025-02-11 | Martin       | Tea             |
| 2025-02-12 | Martin       | Small           |
| 2025-02-12 | Omar         | Small           |
| 2025-02-12 | Omar         | Large           |
| 2025-02-19 | Martin       | Small           |

# Constants in Select

```
-- constants
select productname,
'is good for you' as advertisement
from product;
```

Result:

| A-Z productname ▼ | A-Z advertisement ▼ |
|-------------------|---------------------|
| Tea               | is good for you     |
| Small             | is good for you     |
| Large             | is good for you     |

Product (ProductName, Price)

| ProductName | Price |
|-------------|-------|
| Tea         | 100   |
| Small       | 85    |
| Large       | 100   |

This is a constant.  
This can be a string  
(like here), an int, or  
any other data type.

# Select Distinct

The DISTINCT clause is used to return only the unique rows in the result. Duplicate rows are filtered out.

```
-- distinct cashiers  
select distinct cashier  
from shift;
```

| A-Z cashier ▼ |
|---------------|
| Mads          |
| Anna          |

```
-- distinct purchase by product  
and user  
select distinct productname,  
username  
from purchase;
```

| A-Z productname ▼ | A-Z username ▼ |
|-------------------|----------------|
| Large             | Omar           |
| Tea               | Martin         |
| Small             | Martin         |
| Small             | Omar           |

# Live Exercise 1

What do the following queries return?

```
select price  
from product;
```



| 123 price |
|-----------|
| 100       |
| 85        |
| 100       |

```
select 100 as price  
from product;
```



| 123 price |
|-----------|
| 100       |
| 100       |
| 100       |

```
select 100 as price;
```



| 123 price |
|-----------|
| 100       |

Product (ProductName, Price)

| ProductName | Price |
|-------------|-------|
| Tea         | 100   |
| Small       | 85    |
| Large       | 100   |

# Live Exercise 2

Coffeehouse has a 50% discount on a second product of the same type. For each product, output the price of two ('priceOfTwo').

Product (ProductName, Price)

| ProductName | Price |
|-------------|-------|
| Tea         | 100   |
| Small       | 85    |
| Large       | 100   |

| A-Z productname ▼ | 123 price ▼ | 123 priceOfTwo ▼ |
|-------------------|-------------|------------------|
| Tea               | 100         | 150              |
| Small             | 85          | 127.5            |
| Large             | 100         | 150              |

```
select *, price * 1.5 as priceOfTwo
from product;
```

# Where Clause

The **where** clause specifies filters to apply to the data. This allows to select only a subset of the data.

- Comparisons =, <>, <, >, <=, >= and other operators that produce Boolean-valued results.
- Boolean operators **AND**, **OR**, **NOT**

```
-- price of a tea  
select *  
from product  
where productname = 'Tea';
```



| A-Z productname ▼ | 123 price ▼ |
|-------------------|-------------|
| Tea               | 100         |

# NULL Values

**NULL** values are special values that are used to represent missing data in SQL.<sup>1</sup>

- Columns of any type can contain **NULL** values.
- Logically, a **NULL** value can be seen as “the value of this field is unknown”.

```
-- price of 'fancy' product  
-- is unknown
```

```
insert into product  
  values('Fancy', null);  
select * from product;
```



| A-Z productname ▼ | 123 price ▼ |
|-------------------|-------------|
| Tea               | 100         |
| Small             | 85          |
| Large             | 100         |
| Fancy             | [NULL]      |

# NULL Values and Boolean Logic

Boolean logic is ternary in SQL

AND

|       | true  | false | null  |
|-------|-------|-------|-------|
| true  | true  | false | null  |
| false | false | false | false |
| null  | null  | false | null  |

OR

|       | true | false | null |
|-------|------|-------|------|
| true  | true | true  | true |
| false | true | false | null |
| null  | true | null  | null |

NOT

|       | NOT   |
|-------|-------|
| true  | false |
| false | true  |
| null  | null  |

# NULL Values

Test with **is null** and **is not null**

Product (ProductName, Price)

| ProductName | Price |
|-------------|-------|
| Tea         | 100   |
| Small       | 85    |
| Large       | 100   |
| Fancy       | NULL  |

-- products with known price

```
select *  
from product  
where price is not null;
```



| A-Z productname ▼ | 123 price ▼ |
|-------------------|-------------|
| Tea               | 100         |
| Small             | 85          |
| Large             | 100         |

-- products with unknown price

```
select *  
from product  
where price is null;
```



| A-Z productname ▼ | 123 price ▼ |
|-------------------|-------------|
| Fancy             | [NULL]      |

# NULL Values

**coalesce** is a helpful function in practice to convert nulls to another value, such as a default string or number.

```
-- convert null to a number  
select *, coalesce(price, 110) as price_with_default  
from product;
```

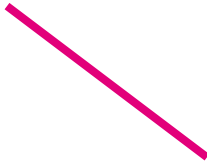
| A-Z productname ▼ | 123 price ▼ | 123 price_with_default ▼ |
|-------------------|-------------|--------------------------|
| Tea               | 100         | 100                      |
| Small             | 85          | 85                       |
| Large             | 100         | 100                      |
| Fancy             | [NULL]      | 110                      |

# IN Operator

The **IN** operator checks containment of the left expression inside the set of expressions on the right.

```
-- coffee products  
select *  
from product  
where productname in ('Small', 'Large');
```

| A-z productname ▼ | 123 price ▼ |
|-------------------|-------------|
| Small             | 85          |
| Large             | 100         |



Can also be a nested query. More on that later.

# Pattern Matching

The **LIKE** operator returns true if the string matches the supplied pattern.

- Pattern is a string with:
  - % matching any string
  - \_ matching any character
- **NOT LIKE** returns false if **LIKE** returns true

```
-- user names starting with 'A'  
select *  
from user  
where username like 'A%';
```



| A-Z username ▼ |
|----------------|
| Anna           |

# Sorting

So far, all results were not guaranteed to be ordered.  
Ordering can be achieved by the **order by** clause.

- **asc** sorts results in ascending order (lowest first)
- **desc** sorts results in descending (highest first)
- Sorting by multiple attributes is possible

```
-- sort purchases by user and by  
-- time, newest purchase first  
select username,  
        purchasetime,  
        productname  
from purchase  
order by username, purchasetime desc;
```

| A-Z username ▼ | 🕒 purchasetime ▼        | A-Z productname |
|----------------|-------------------------|-----------------|
| Martin         | 2025-02-19 09:00:00.000 | Small           |
| Martin         | 2025-02-12 10:03:00.000 | Small           |
| Martin         | 2025-02-11 09:55:00.000 | Tea             |
| Omar           | 2025-02-12 10:06:00.000 | Large           |
| Omar           | 2025-02-12 10:05:00.000 | Small           |

Multiple attributes

# Limit

In practice, it's often helpful to output the top N results. This can be achieved with **order by** and **limit**.

```
-- output the last two purchases by Martin
select *
from purchase
where username = 'Martin'
order by purchasetime desc
limit 2;
```

|  purchasetime ▼ | A-Z productname ▼ | A-Z username ▼ |
|--------------------------------------------------------------------------------------------------|-------------------|----------------|
| 2025-02-19 09:00:00.000                                                                          | Small             | Martin         |
| 2025-02-12 10:03:00.000                                                                          | Small             | Martin         |

# Live Exercise 3

What is the cheapest product?

(Note: Only works because there is a single cheapest product. We will solve this differently later, in Live Exercise 5.)

```
select *  
from product  
order by price  
limit 1;
```

| A-Z productname ▼ | 123 price ▼ |
|-------------------|-------------|
| Small             | 85          |

**Product** (ProductName, Price)

| ProductName | Price |
|-------------|-------|
| Tea         | 100   |
| Small       | 85    |
| Large       | 100   |
| Fancy       | NULL  |

# Live Exercise 4

-- what is the result of this query

```
select (true and true) as a,  
       (true and false) as b,  
       (true and null) as c,  
       (true or true) as d,  
       (true or false) as e,  
       (true or null) as f;
```

| a    | b     | c    | d    | e    | f    |
|------|-------|------|------|------|------|
| true | false | null | true | true | true |

# Agenda

## Select

- Renaming
- Expressions, casts, constants
- Distinct
- Filtering rows (**where**)
- Null values, **IN**, **LIKE**
- Sorting and limit queries

## Aggregation

- Aggregate Functions
- Grouping and filtering groups (**having**)

## Joins

- Semantics and syntax

# Aggregations

So far: per-row computations and filtering

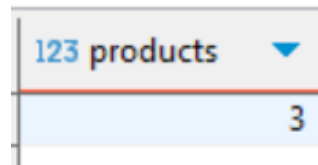
Now: computations over multiple rows = aggregations

# Aggregate Functions

Most common aggregate functions in SQL include:

- COUNT() Returns the **number of rows**
- SUM() Returns the **total sum** of a numeric column
- AVG() Returns the **average** value of a numeric column
- MIN() Returns the **minimum** value in a column
- MAX() Returns the **maximum** value in a column

```
-- how many products did Martin buy  
select count(productname) as products  
from purchase  
where username = 'Martin';
```



|                |
|----------------|
| 123 products ▼ |
| 3              |

In practice very often:  
count(\*)

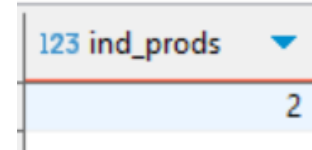
# Aggregate + DISTINCT

When the **DISTINCT** clause is provided, only distinct values are considered in the computation of the aggregate.<sup>1</sup>

**Purchase** (PurchaseTime, ProductName, UserName)

| PurchaseTime  | ProductName | UserName |
|---------------|-------------|----------|
| Feb 11, 9:55  | Tea         | Martin   |
| Feb 12, 10:03 | Small       | Martin   |
| Feb 12, 10:05 | Small       | Omar     |
| Feb 12, 10:06 | Large       | Omar     |
| Feb 19, 9:00  | Small       | Martin   |

```
-- how many individual products
-- did Martin buy
select count(distinct productname)
       as ind_prods
from purchase
where username = 'Martin';
```



|               |   |
|---------------|---|
| 123 ind_prods | ▼ |
| 2             |   |

# NULLs in Aggregations

NULL values are generally ignored. Nulls never contribute to sum, average, count, and are never the min or max.

If there are **only** NULL values, the result of the aggregation is null. Exception: count of an empty is set is 0.

**Product** (ProductName, Price)

| ProductName | Price |
|-------------|-------|
| Tea         | 100   |
| Small       | 85    |
| Large       | 100   |
| Fancy       | NULL  |

-- nulls are ignored in aggregates

```
select count(*),  
       count(price),  
       min(price),  
       max(price)
```

```
from product;
```

| 123 count_star() | 123 count(price) | 123 min(price) | 123 max(price) |
|------------------|------------------|----------------|----------------|
| 4                | 3                | 85             | 100            |

# Grouping-Based Aggregations

Select-from-where followed by group by:

```
select      -- list of attributes  
from        -- one or more relations  
where       -- filters  
group by    -- list of attributes
```

The result is a relation that is:

- Grouped according to the values of the attributes in the **group by** clause
- Any aggregation is applied only within each group

# Group By Examples

Purchase (PurchaseTime, ProductName, UserName)

| PurchaseTime  | ProductName | UserName |
|---------------|-------------|----------|
| Feb 11, 9:55  | Tea         | Martin   |
| Feb 12, 10:03 | Small       | Martin   |
| Feb 12, 10:05 | Small       | Omar     |
| Feb 12, 10:06 | Large       | Omar     |
| Feb 19, 9:00  | Small       | Martin   |

```
-- how many products did each customer buy,  
-- output most frequent customers first  
select username,count(*) as count  
from purchase  
group by username  
order by count desc;
```

| A-Z username ▼ | 123 count ▼ |
|----------------|-------------|
| Martin         | 3           |
| Omar           | 2           |

```
-- list products by popularity (i.e. how  
-- often they were bought)  
select productname,count(*) as popularity  
from purchase  
group by productname  
order by popularity desc, productname;
```

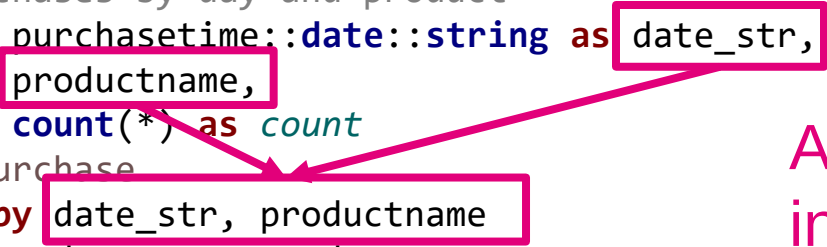
| A-Z productname ▼ | 123 popularity ▼ |
|-------------------|------------------|
| Small             | 3                |
| Large             | 1                |
| Tea               | 1                |

# SELECT Attributes with Aggregations

If any aggregation is used, then each element of the **select** list must be either:

- An attribute in the **group by** list, or
- An aggregation (**count**, **min**, **max**, ...)

```
-- purchases by day and product
select purchasetime::date::string as date_str,
       productname,
       count(*) as count
from purchase
group by date_str, productname
order by date_str, productname;
```



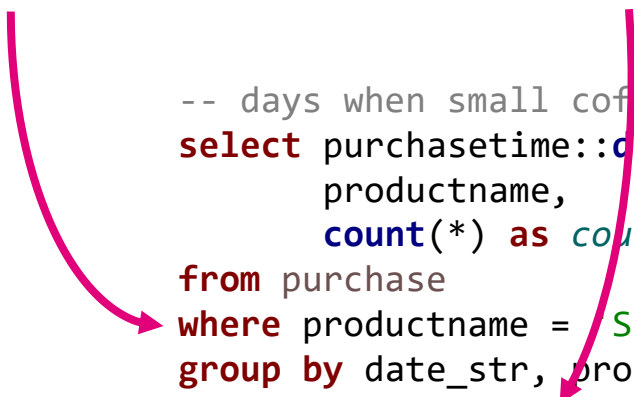
Attributes in select must be in the group by list.

| A-Z date_str ▼ | A-Z productname ▼ | 123 count ▼ |
|----------------|-------------------|-------------|
| 2025-02-11     | Tea               | 1           |
| 2025-02-12     | Large             | 1           |
| 2025-02-12     | Small             | 2           |
| 2025-02-19     | Small             | 1           |

# HAVING Clause

**Having <condition>** may follow a **group by** clause.

- The condition applies to each group. Groups not satisfying the condition are eliminated.
- **where** filters rows, **having** filters groups.



```
-- days when small coffees were sold at least twice
select purchasetime::date::string as date_str,
       productname,
       count(*) as count
from purchase
where productname = 'Small'
group by date_str, productname
having count >= 2;
```

| A-Z date_str ▼ | A-Z productname ▼ | 123 count ▼ |
|----------------|-------------------|-------------|
| 2025-02-12     | Small             | 2           |

# Subqueries / Nested Queries

Subqueries (or nested queries) are parenthesized query expressions that appear as part of a larger, outer query<sup>1</sup>.

```
-- purchases of products that cost 100 DKK
-- (note: we will solve this differently later)
select *
from purchase
where productname in
    (select productname from product where price = 100);
```

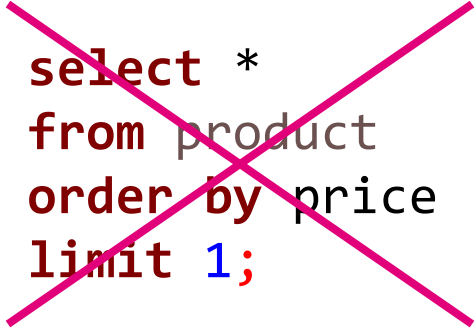
|  purchasetime | A-Z productname | A-Z username |
|------------------------------------------------------------------------------------------------|-----------------|--------------|
| 2025-02-11 09:55:00.000                                                                        | Tea             | Martin       |
| 2025-02-12 10:06:00.000                                                                        | Large           | Omar         |

# Live Exercise 5

Product (ProductName, Price)

What are the cheapest products?  
What are the most expensive products?

| ProductName | Price |
|-------------|-------|
| Tea         | 100   |
| Small       | 85    |
| Large       | 100   |
| Fancy       | NULL  |



```
select *  
from product  
order by price  
limit 1;
```

Solution from live exercise 3  
does not work in all cases.  
Let's do it correctly!

# Live Exercise 5

**Product** (ProductName, Price)

What are the cheapest products?  
What are the most expensive products?

| ProductName | Price |
|-------------|-------|
| Tea         | 100   |
| Small       | 85    |
| Large       | 100   |
| Fancy       | NULL  |

```
-- cheapest products
select *
from product
where price =
  (select min(price) from product)
order by productname;
```

| A-Z productname ▼ | 123 price ▼ |
|-------------------|-------------|
| Small             | 85          |

```
-- most expensive products
select *
from product
where price =
  (select max(price) from product)
order by productname;
```

| A-Z productname ▼ | 123 price ▼ |
|-------------------|-------------|
| Large             | 100         |
| Tea               | 100         |

# Live Exercise 6

Describe what this query is doing and list its output.

```
select *  
from purchase  
where purchasetime::date in (  
    select purchasetime::date as date  
    from purchase  
    group by date  
    having count(*) = 1  
);
```

Solution: Purchases made on days with only one transaction.

Purchase (PurchaseTime, ProductName, UserName)

| PurchaseTime  | ProductName | UserName |
|---------------|-------------|----------|
| Feb 11, 9:55  | Tea         | Martin   |
| Feb 12, 10:03 | Small       | Martin   |
| Feb 12, 10:05 | Small       | Omar     |
| Feb 12, 10:06 | Large       | Omar     |
| Feb 19, 9:00  | Small       | Martin   |

|  purchasetime | A-Z productname | A-Z username |
|--------------------------------------------------------------------------------------------------|-----------------|--------------|
| 2025-02-11 09:55:00.000                                                                          | Tea             | Martin       |
| 2025-02-19 09:00:00.000                                                                          | Small           | Martin       |

# Agenda

## Select

- Renaming
- Expressions, casts, constants
- Distinct
- Filtering rows (**where**)
- Null values, **IN**, **LIKE**
- Sorting and limit queries

## Aggregation

- Aggregate Functions
- Grouping and filtering groups (**having**)

## Joins

- Semantics and syntax

# Multiple Relations

So far: only one relation per query

Now: queries spanning multiple relations → joins

# Multi-Relation Queries

Queries combine data from more than one relation

- Several tables in the **from** clause
- Distinguish attributes of the same name by **<relation>.<attribute>**, renaming tables helpful

```
-- purchases with email addresses
```

```
select purchase.*, email
```

```
from purchase, user
```

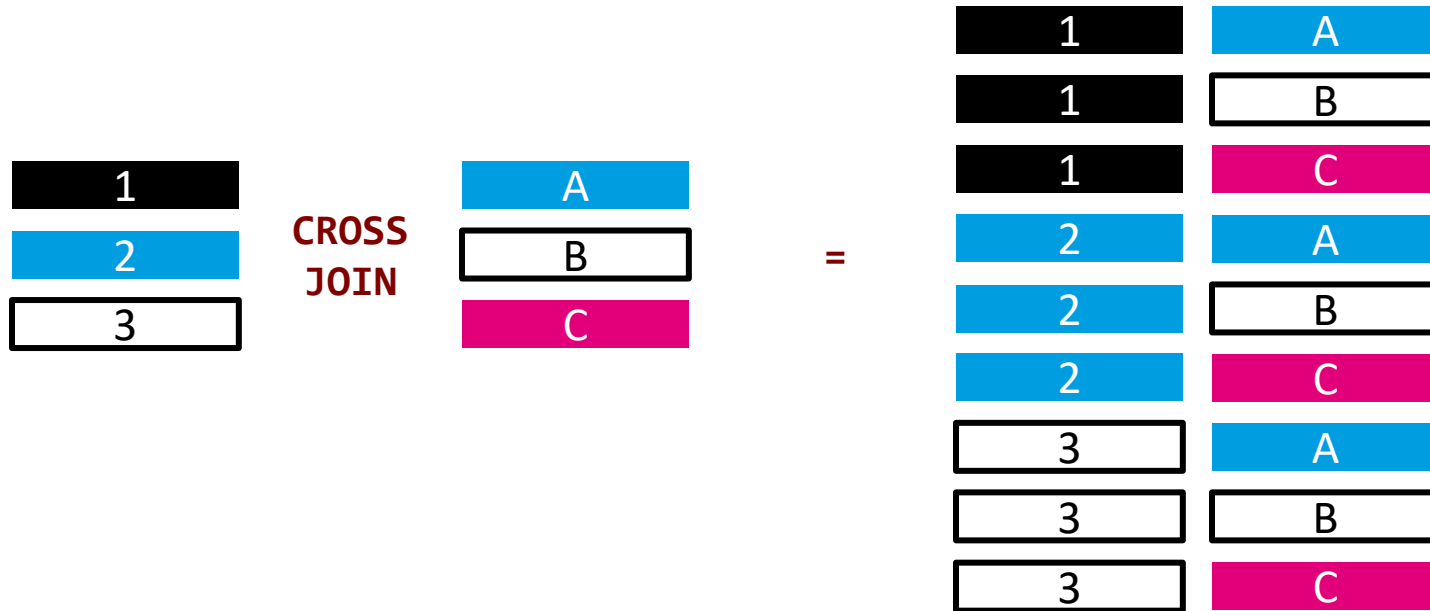
```
where purchase.username = user.username;
```

This is a join!

|  purchasetime ▼ | A-Z productname ▼ | A-Z username ▼ | A-Z email ▼ |
|--------------------------------------------------------------------------------------------------|-------------------|----------------|-------------|
| 2025-02-11 09:55:00.000                                                                          | Tea               | Martin         | mhent       |
| 2025-02-12 10:03:00.000                                                                          | Small             | Martin         | mhent       |
| 2025-02-12 10:05:00.000                                                                          | Small             | Omar           | omsh        |
| 2025-02-12 10:06:00.000                                                                          | Large             | Omar           | omsh        |
| 2025-02-19 09:00:00.000                                                                          | Small             | Martin         | mhent       |

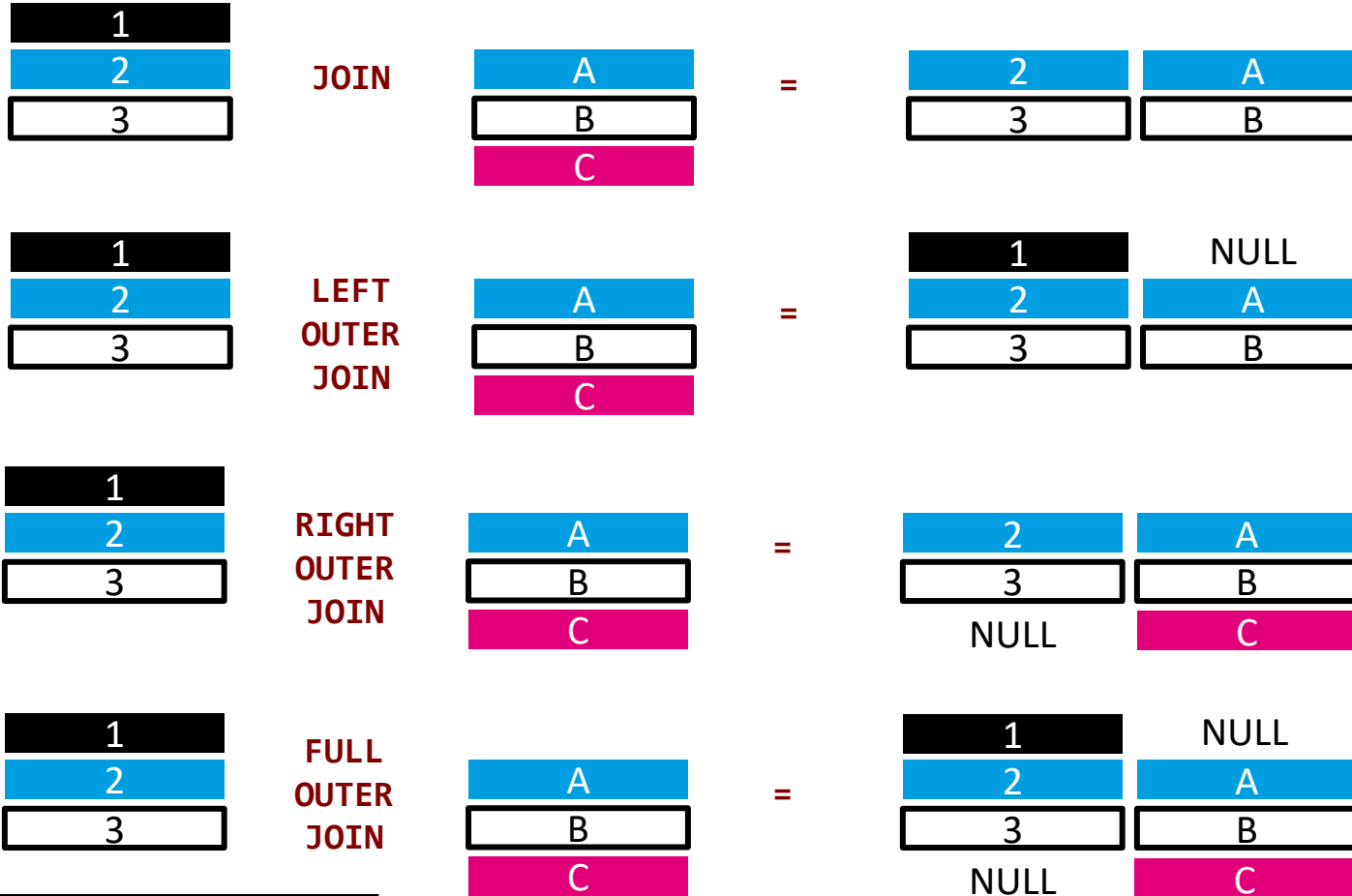
# Joins

First of all, let's look at **CROSS JOIN** first, because all other join types can be derived from **CROSS JOIN**<sup>1</sup>.



# Joins

All other joins are based on cross joins, but with additional filters, and perhaps unions of rows that did not match<sup>1</sup>. Join on color attribute:



# Join Syntax

## Two main syntax for inner joins:

```
-- "join" syntax
select purchase.*, email
from purchase join user
on purchase.username = user.username;
```

```
-- equivalent "cross join" syntax
select purchase.*, email
from purchase, user
where purchase.username = user.username;
```

|  purchasetime | A-Z productname | A-Z username | A-Z email |
|-------------------------------------------------------------------------------------------------|-----------------|--------------|-----------|
| 2025-02-11 09:55:00.000                                                                         | Tea             | Martin       | mhent     |
| 2025-02-12 10:03:00.000                                                                         | Small           | Martin       | mhent     |
| 2025-02-12 10:05:00.000                                                                         | Small           | Omar         | omsh      |
| 2025-02-12 10:06:00.000                                                                         | Large           | Omar         | omsh      |
| 2025-02-19 09:00:00.000                                                                         | Small           | Martin       | mhent     |

## Outer joins must be written using the “join” syntax:

```
-- all users and the products they purchased
select u.username, pu.productname
from user u left outer join purchase pu
on u.username = pu.username
order by u.username;
```

| A-Z username | A-Z productname |
|--------------|-----------------|
| Anna         | [NULL]          |
| Martin       | Tea             |
| Martin       | Small           |
| Martin       | Small           |
| Omar         | Small           |
| Omar         | Large           |

# Joins and Aggregations

Of course, all of this can be combined. For example, joins and aggregations.

```
-- revenue per day
select purchasetime::date as day, sum(price) as revenue
from purchase join product
on purchase.productname = product.productname
group by day
order by day;
```

|  day ▼ | 123 revenue ▼ |
|-----------------------------------------------------------------------------------------|---------------|
| 2025-02-11 00:00:00.000                                                                 | 100           |
| 2025-02-12 00:00:00.000                                                                 | 270           |
| 2025-02-19 00:00:00.000                                                                 | 85            |

# Takeaways

THAT'S IMPOSSIBLE!



NOT MUCH EASIER...



Dataedo /cartoon

IT UNIVERSITY OF COPENHAGEN

Piotr@Dataedo

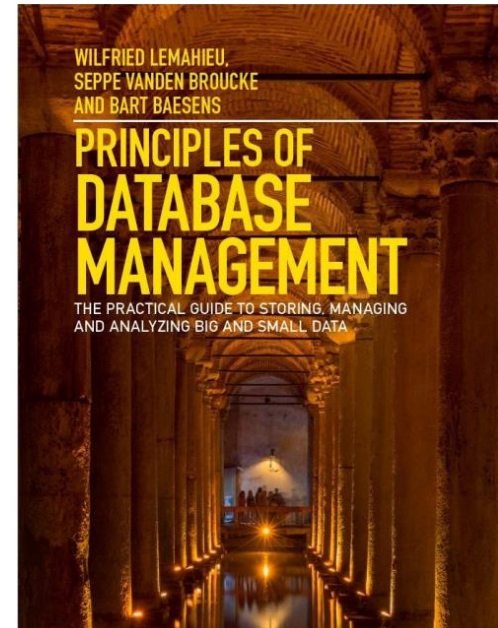
# Actual Takeaways

SQL is a powerful language to analyze and modify data.

Remember:

- Results are not sorted, use **order by**
- SQL boolean logic is ternary because of nulls
- **where** filters rows, **having** filters groups
- Finding min/max entities often needs subqueries (see live exercise 5)

Readings: Chapter 7.3



and [Say NO to Venn Diagrams When Explaining JOINS](#)  
and [SQL Introduction – DuckDB](#) (as a reference)